

Plan de Pruebas

Versión <1.0>

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Histórico de revisiones

Fecha	Versión	Descripción	Autor
<03/12/2025>	<1.0>	<Plan de pruebas implementado>	<Dimas Antonio Torralba Raluy>
<07/12/2025>	<1.1>	<Complementado plan de pruebas>	<Yoel Casamayor Sahuquillo>

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Índice

1.	Introducción	1
1.1	Objetivos	1
1.2	Ámbito	1
1.3	Referencias	1
2.	Elementos de prueba	2
3.	Características que se deben probar	2
4.	Características que no se van a probar	2
5.	Aproximación	3
5.1	Pruebas funcionales	3
5.2	Pruebas de carga	3
5.3	Pruebas de rendimiento	3
5.4	Pruebas de seguridad	4
6.	Criterios para decidir si pasa/falla un elemento	4
7.	Criterios de suspensión y reanudación	4
7.1	Suspensión	4
7.2	Reanudación	4
8.	Pruebas que se deben entregar	4
9.	Tareas de prueba	6
10.	Necesidades ambientales	6
10.1	Hardware	6
10.2	Software	6
10.3	Seguridad	7
10.4	Herramientas	7
10.5	Publicaciones	7
11.	Responsabilidades.	7
12.	Aprobación	7

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Plan de Pruebas

1. Introducción

Este documento describe el plan de pruebas para el sistema de gestión de la ferretería "Los Guatios". El enfoque principal es validar la lógica de negocio y los puntos finales (endpoints) de la API REST desarrollada en .NET Core.

1.1 Objetivos

Verificar que los controladores de la API gestionan correctamente las peticiones de los clientes (Alquileres, Compras, Reparaciones y Ofertas).

Asegurar el cumplimiento de las reglas de negocio descritas en los Casos de Uso (ej. validación de stock, fechas futuras, formatos de dirección).

Garantizar que los filtros de búsqueda devuelven los resultados esperados.

1.2 Ámbito

Las pruebas abarcan las Pruebas Unitarias y de Integración de Componentes del Backend.

Módulos incluidos: Gestión de Alquileres, Compras, Herramientas, Ofertas y Reparaciones.

Base de Datos: Se utiliza una base de datos en memoria (SQLite) para simular la persistencia de datos durante los tests.

1.3 Referencias

Documento de requisitos: CasosDeUso_grupoA1.pdf.

Estándar IEEE 829 para documentación de pruebas de software.

Código fuente de los tests implementados en la solución AppForSEII2526.

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

2. Elementos de prueba

Los elementos de software que serán sometidos a prueba son los Controladores de la API y sus respectivos DTOs de transferencia de datos:

AlquileresController: Métodos CreateAlquiler y GetDetalleHerramientasParaAlquiler.

ComprasController: Métodos CrearCompra y GetDetalleHerramientasParaCompra.

HerramientasController: Métodos de filtrado (GetHerramientasParaCompra, GetHerramientasParaAlquiler, GetHerramientasParaOferta, GetHerramientasParaReparacion)

OfertasController: Métodos CrearOferta y GetDetalleHerramientasParaOferta.

ReparacionesController: Métodos CreateReparacion y GetDetalleHerramientasParaReparación

3. Características que se deben probar

Se validarán las siguientes funcionalidades críticas basadas en los requisitos del negocio:

- Validación de Datos de Entrada:
 - Compras: Verificar stock suficiente, cantidades positivas y existencia de usuario.
 - Alquileres: Validar que la dirección comience por "Calle" y fechas coherentes.
 - Reparaciones: Validar formato de teléfono (+34) y fechas futuras para entrega/recogida.
 - Ofertas: Validar porcentajes de descuento (1-90%) y duración mínima de una semana.
- Cálculo de Precios:

Verificar que el total se calcula correctamente sumando los items y aplicando descuentos o tarifas por día.
- Filtrado de Herramientas:

Comprobar que los filtros por nombre, material, precio y fabricante devuelven la lista exacta de herramientas esperadas.
- Persistencia:

Asegurar que los datos se guardan correctamente en la base de datos en memoria y se pueden recuperar mediante los endpoints de detalle.

4. Características que no se van a probar

Interfaz de Usuario (Frontend): Las pruebas se limitan a la API y la lógica de backend.

Concurrencia y Rendimiento: Al usar base de datos en memoria, no se evaluará el rendimiento bajo carga real ni bloqueos de base de datos.

Pasarelas de Pago: Se simulan los métodos de pago, pero no se conecta con servicios bancarios reales.

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Riesgo Técnico: El entorno de pruebas utiliza Microsoft.EntityFrameworkCore.Sqlite en memoria. Existe el riesgo de que comportamientos específicos de SQL (como la sensibilidad a mayúsculas/minúsculas o tipos de fecha) difieran del entorno de producción (SQL Server/MySQL). Se recomienda una prueba de humo manual en el entorno de pre-producción.

5. Aproximación

Se utiliza una estrategia de Pruebas Automatizadas utilizando el framework xUnit.

Técnica: Caja Blanca (conocemos el código) y Caja Negra (probamos la entrada/salida).

Patrón: Arrange-Act-Assert (Preparar entorno, Ejecutar acción, Verificar resultado).

Inyección de Dependencias: Se utiliza Mock para simular el Logger y un contexto de base de datos SQLite en memoria (AppForSEII25264SqliteUT) para aislar las pruebas.

Casos Borde: Se utiliza [Theory] y [MemberData] para inyectar múltiples escenarios de error (datos inválidos, nulos, límites) y asegurar que la API responde con BadRequest.

5.1 Pruebas funcionales

Estrategia: Se realizarán pruebas automatizadas de interfaz de usuario (UI) utilizando Selenium WebDriver.

Herramienta: Se ha adaptado el proyecto base u.weit (framework de automatización en Java/.NET) para interactuar con los elementos específicos de la web "Los Guatios".

Objetivo: Verificar que los flujos críticos de negocio funcionan desde la perspectiva del usuario final en un navegador real. Esto incluye:

- Navegación por el catálogo de herramientas.
- Interacción con el carrito de compra (añadir/eliminar items).
- Cumplimentación de formularios de Alquiler y Reparación.
- Validación de mensajes de error en pantalla (pop-ups, textos de validación)

5.2 Pruebas de carga

Estrategia: Se utilizará Apache JMeter para simular una alta concurrencia de usuarios virtuales accediendo a la API REST.

Configuración del Test (Test Plan):

- Grupo de Hilos (Thread Group): Se configurará un escenario escalonado (*Ramp-up*) que inicie con 10 usuarios y aumente progresivamente hasta 100 usuarios simultáneos en un periodo de 60 segundos.
- Peticiones: Se realizarán peticiones HTTP (GET y POST) contra los controladores críticos identificados: HerramientasController (Catálogo) y OfertasController (Creación masiva).

Objetivo: Identificar el "punto de ruptura" (Break Point) del servidor, es decir, el momento en que el sistema empieza a devolver errores HTTP 500 o 503 debido a la saturación de conexiones.

5.3 Pruebas de rendimiento

Estrategia: Se analizarán los informes generados por los "Listeners" de JMeter (View Results Tree, Summary Report) durante las pruebas de carga.

Métricas Clave:

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

- Throughput (Caudal): Número de peticiones por segundo que la API es capaz de procesar.
- Tiempo de Latencia: Tiempo que tarda el primer byte en llegar al cliente.
- % de Error: Debe mantenerse en 0% bajo carga normal y no superar el 5% bajo carga de estrés.

Objetivo: Asegurar que los tiempos de respuesta del controlador ComprasController se mantienen por debajo de los 2000ms (2 segundos) incluso con 50 usuarios concurrentes operando en el sistema.

5.4 Pruebas de seguridad

Se valida que no se puedan realizar operaciones con usuarios inexistentes o IDs inválidos

6. Criterios para decidir si pasa/falla un elemento

Pasa:

El código de estado HTTP es el esperado (200, 201, 400, 404).

Los valores devueltos en el DTO coinciden exactamente con los esperados (Assert.Equal).

No se producen excepciones no controladas (500 Internal Server Error).

Falla:

Cualquier discrepancia en los cálculos de precios o fechas.

Si se permite guardar un registro que incumple una regla de validación (ej. guardar una compra con stock insuficiente).

7. Criterios de suspensión y reanudación

Suspensión: Las pruebas se suspenderán si existen errores de compilación en la solución o si el contexto de la base de datos en memoria no puede inicializarse.

Reanudación: Una vez corregidos los errores de sintaxis o configuración, se re-ejecutará la suite completa de pruebas.

7.1 Suspensión

7.2 Reanudación

8. Pruebas que se deben entregar

El conjunto de pruebas entregable consta de los siguientes archivos de código fuente:

Requisito (Caso de Uso)	Funcionalidad / Flujo	Archivo de Prueba (Unit Test)
CU 1. Comprar	1. Selección/Filtrado: Buscar herramientas por material/precio.	GetSeleccionParaCompras_test.cs

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Requisito (Caso de Uso)	Funcionalidad / Flujo	Archivo de Prueba (Unit Test)
CU Comprar 1.	2. Acción: Realizar la compra (Validar stock, usuario).	PostParaCompra_test.cs
CU Comprar 1.	3. Detalle: Ver el resumen de una compra realizada.	GetDetalleParaCompra_test.cs
CU Reparar 2.	1. Selección/Filtrado: Buscar por tiempo de reparación/nombre.	GetHerramientasReparacion_test.cs
CU Reparar 2.	2. Acción: Solicitar reparación (Validar fechas, teléfono).	PostParaReparacion_test.cs
CU Reparar 2.	3. Detalle: Ver el resumen de la reparación.	GetDetalleReparacion_test.cs
CU Ofertas 3.	1. Selección/Filtrado: Buscar herramientas para ofertar (Fabricante).	GetSeleccionParaOferta_test.cs
CU Ofertas 3.	2. Acción: Crear oferta (Validar %, fechas, tipo usuario).	Post_oferta_test.cs

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Requisito (Caso de Uso)	Funcionalidad / Flujo	Archivo de Prueba (Unit Test)
CU 3. Ofertas	3. Detalle: Ver el resumen de la oferta creada.	GetDetalleParaOferta_test.cs
CU 4. Alquilar	1. Selección/Filtrado: Buscar herramientas por nombre/material.	GetHerramientasParaAlquiler_test.cs
CU 4. Alquilar	2. Acción: Realizar alquiler (Validar dirección "Calle").	CreateAlquiler_test.cs
CU 4. Alquilar	3. Detalle: Ver el resumen del alquiler.	GetDetalleParaAlquiler_test.cs

9. Tareas de prueba

Configuración del Entorno: Restaurar paquetes NuGet y compilar la solución.

Ejecución: Ejecutar todos los tests mediante el Explorador de Pruebas de Visual Studio.

Análisis: Revisar los resultados. Si algún test falla, analizar el mensaje de aserción (Assert.Equal failed) y corregir el código del controlador o el test.

Reporte: Documentar los bugs encontrados si la lógica de negocio no se cumple.

10. Necesidades ambientales

10.1 Hardware

PC de desarrollo estándar capaz de ejecutar Visual Studio 2022 o VS Code.

10.2 Software

.NET Core SDK (versión compatible con el proyecto).

Framework de pruebas xUnit.

	Versión: <1.0>
Plan de Pruebas	Fecha: <07/12/2025>
<document identifier>	

Librería Microsoft.EntityFrameworkCore.Sqlite.

Librería Moq.

Apache JMeter 5.x: Para ejecución y análisis de pruebas de rendimiento y carga.

Java Development Kit (JDK): Necesario para ejecutar JMeter.

10.3 Seguridad

Visual Studio 2022 / VS Code.

Git para control de versiones.

10.4 Herramientas

Visual Studio 2022

Git

GitHub

10.5 Publicaciones

11. Responsabilidades.

Desarrolladores:

Implementar el código de la API y escribir los tests unitarios correspondientes.

QA / Tester:

Ejecutar la suite de pruebas, verificar la cobertura de los casos de uso y reportar incidencias.

12. Aprobación

El plan se considera aprobado cuando el 100% de los tests unitarios diseñados ("Happy Path" y casos de error) se ejecutan exitosamente (Verde) en el entorno de integración continua o local.